
《大学计算机基础》实验指导书

机器学习基础

1. 简介

机器学习是一种人工智能领域的技术，通过从数据中学习模式和规律，使计算机能够进行预测和决策。本次实验我们将实践几种常见的机器学习算法，包括线性回归、支持向量机（SVM）、K 均值聚类（Kmeans）和主成分分析（PCA）。通过这些实验，你将学习如何应用这些算法来解决不同类型的问题，并深入了解它们在日常生活中的应用。

2. 实验目的

- 1) 理解不同机器学习算法的基本原理。
- 2) 学会如何使用 Python 编程语言实现这些算法。
- 3) 尝试使用常见的机器学习算法解决日常生活中的问题

3. 实验内容

本次实验分为四个实验任务，需要同学们基于四个经典的机器学习算法完成一些简单的分类、回归任务。

实验任务 2-1 线性回归

实验要求

在线性回归实验中，我们将使用学习时间预测学生成绩（数据文件为 data.xlsx）。通过建立一个一元线性回归模型，我们想尝试找到学习与成绩之间的关系，并使用模型进行预测。

实验运行后，我们将得到一个散点图，其中展示了实际学习时间 time 与成绩 score 的分布，以及使用线性回归模型预测的结果曲线。同时，请你计算该模型的均方根误差（RMSE）以评价效果。

实验参考结果

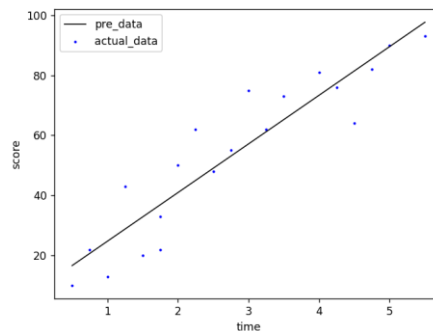


图 2-1

实验提示

(1) 创建一个线性回归模型需要先导入相应的模块

```
from sklearn.linear_model import LinearRegression
```

接着, 你可以使用如下方法可以创建一个线性回归模型并训练, 从而获取一元线性回归模型的参数 k 和 b

```
time = time.reshape(-1, 1) #将学习时间转化为列向量
```

```
model = LinearRegression()
```

```
model.fit(time, score)
```

```
b = model.intercept_ #获得截距
```

```
k = model.coef_ #获得回归系数
```

(2) 以下给出了一个示例, 可计算出模型的 RMSE

```
from sklearn.metrics import mean_squared_error
```

```
rmse=np.sqrt(mean_squared_error(score,pre_score)) #score, pre_score 分别表示实际分数与模型预测的分数
```

(3) 回归结果的显示可采用散点图的方式, 以下给出了一个示例

```
plt.scatter(time, score, color="b", label="actual_data", s=2)
```

```
plt.plot(time, pre_score, color="black", linewidth=1, label="pre_data")
```

实验任务 2-2 SVM 分类

实验要求

鸢尾花数据集 (Iris) 是一个经典的分类实验数据集, 它包含 150 个样本 (数据集的行)。数据集包含 4 个属性 (数据集的列): Sepal Length (花萼长度), Sepal Width (花萼宽度), Petal Length (花瓣长度), Petal Width (花瓣宽度)。在支持向量机 (SVM) 实验中, 我们将使用鸢尾花数据集进行分类, 通过其中一些属性预测鸢尾花属于 Setosa, Versicolour, Virginica 三个种类中的哪一类。

希望同学们完成以下实验任务:

- 构建一个 SVM 分类器，将数据集划分为训练集和测试集，使用数据集中提供的属性，分类不同种类的鸢尾花，并给出该模型在测试集上的 F1-score，借助以上实验来了解 SVM 在机器学习中的应用。
- 绘制图表，分别包括鸢尾花花瓣的长度和宽度，花萼的长度和宽度散点图和每个数据对应的类别（这部分可以在分类前绘制）（如图 2-2-1、2-2-2 所示）
- 在两个特征维度（petal 的长宽）下，使用 SVM 分类器绘制的决策边界（图 2-2-3 给出了一个相似的示例）。

参考实验结果

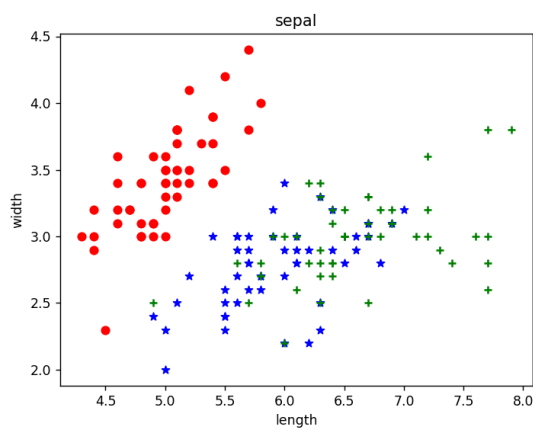


图 2-2-1

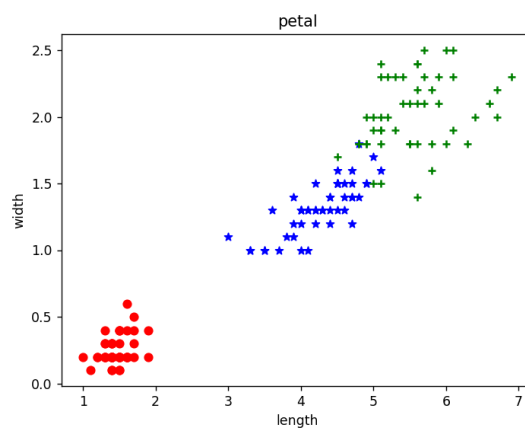


图 2-2-2

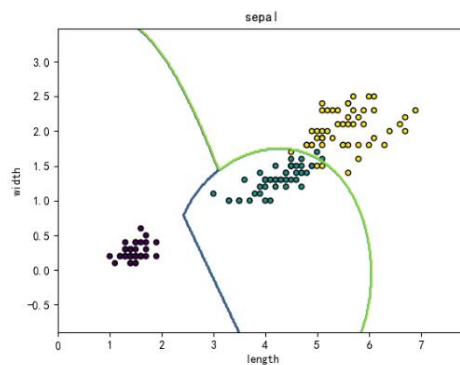


图 2-2-3

实验提示

(1) 以下示例代码可读取鸢尾花数据集，并取出花萼的数据

```
from sklearn.datasets import load_iris
iris = load_iris() # 读取数据集
data = iris.data # 获得特征值
target = iris.target # 获得标签值
print(type(iris), dir(iris))
petal_data = iris.data[:, :2] # 取花萼数据
```

(2) 以下示例可以完成将数据集划分为训练集和测试集，并创建分类器进行训练

```
from sklearn.model_selection import train_test_split
from sklearn import svm
X_train, X_test, y_train, y_test = train_test_split(data, target, test_size=0.1, random_state=0)
clf = svm.SVC(kernel='rbf', C=100) # 创建 SVM 分类器
clf.fit(X_train, y_train) # 使用该分类器进行训练
```

(3) 训练完成之后，你需要利用该模型给出其在测试集上的预测结果，并将其和真实标签对比，给出 F1-score。以下示例代码完成模型 F1-score 的输出。

```
from sklearn.metrics import f1_score
y_pred = clf.predict(X_test) # 用测试集 X_test 预测结果，clf 为训好的模型
print("F-score: {0:.2f}".format(f1_score(y_pred, y_test, average='micro'))) # y_pred, y_test 分别为模型预测值、实际标签值
```

(4) 以下函数可完成决策边界的绘制，参数解释请参见注释内容：

```
def plot_contours(ax, clf, xx, yy, **params):
    """
    在图表上绘制 SVM 模型的决策边界。
    参数解释：
    ax : matplotlib.axes._subplots.AxesSubplot
        绘制图表的坐标轴对象。
    clf : sklearn.svm.SVC
        训练好的 SVM 分类器。
    xx : numpy.ndarray
        横坐标的网格点。
    yy : numpy.ndarray
        纵坐标的网格点。
    **params : dict
        其他参数传递给 ax.contour 函数。
    返回：
    out : matplotlib.contour.QuadContourSet
        绘制的轮廓对象。
    """
    # 使用 SVM 模型对网格点进行预测
    Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
    # 将预测结果重新调整为与网格点形状相同
```

```

Z = Z.reshape(xx.shape)
# 在图表上绘制决策边界轮廓
out = ax.contour(xx, yy, Z, **params)
return out

```

以下给出了该函数的一个使用示例，目的是完成取花瓣（sepal）的长和宽使用 SVM 绘制决策边界，你可参照示例完成你的模型训练和决策边界绘制：

```

sepal_data = iris.data[:, 2:] # 从数据集中取花瓣数据
X0, X1 = sepal_data[:, 0], sepal_data[:, 1] #
xx, yy = make_meshgrid(X0, X1)
ax = plt.subplots()[1]
clf = svm.SVC(kernel='rbf', C=100)
clf.fit(sepal_data, target)
plot_contours(ax, clf, xx, yy, alpha=0.8)

```

（5）以下函数可根据数据范围在画板上绘制网格，在可视化的过程中你可以参照使用。

```

def make_meshgrid(x, y, h=.02):
    x_min, x_max = x.min() - 1, x.max() + 1
    y_min, y_max = y.min() - 1, y.max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))
    return xx, yy

```

实验任务 2-3 K-means 聚类

实验要求：

在本实验中，我们将使用 K 均值聚类算法对一个购物中心的顾客数据进行聚类（数据集为 mall_customer.csv），以了解不同顾客群体的特征和分布情况。我们将使用年龄、年收入和消费得分作为特征，并通过 K 均值聚类算法将顾客划分为不同的群体。

- 在实验中，请根据肘部原理找到合适的参数 [K-means 聚类算法中的 K 如何确定](#)，再进行模型构建。给出一个图形化结果，其中展示了使用不同 K 值（K 取值范围为[2,9]）对顾客进行 K-means 聚类的 loss 值（如图 2-3-1 所示）。
- 给出一个三维的散点图，在其中 K-means 聚类的结果。每个数据点被归类到不同的群体，并在三维空间中显示其年龄、年收入和消费得分的分布情况（如图 2-3-2 所示）。

参考实验结果

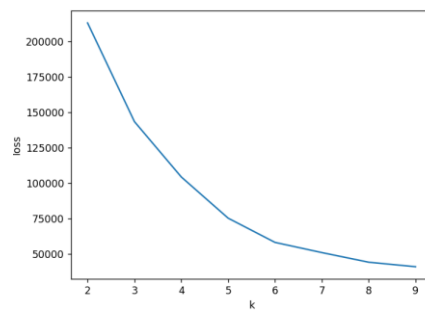


图 2-3-1

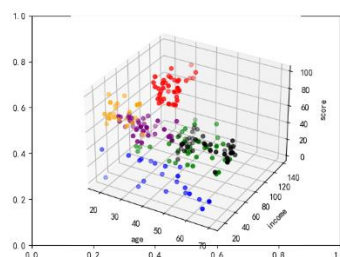


图 2-3-2

实验提示

(1) 以下示例代码可以针对数据集 `data`, 帮助保存聚类时的 `loss` 值尝试在一定的范围内寻找和确定合适的聚类中心数, 寻找最明显的拐点为 `k` 值。注意数据集中“性别”属性值需要先处理成整数 0/1 表示。

```
loss = []
```

```
for i in range(2, 10):
```

```
    model = KMeans(n_clusters=i).fit(data)
```

```
    loss.append(model.inertia_)
```

将 `loss` 值可视化之后, 可以得到如例子所示的损失函数值随 `K` 值的变化可视化结果。

(2) 当确定好 `K` 值之后, 可以使用使用 `kmeans` 算法聚类

```
from sklearn.cluster import KMeans
```

```
kmeans = KMeans(n_clusters=?, init='k-means++')
```

```
kmeans.fit(data)
```

(3) 以下示例代码尝试将聚类结果可视化

```
fig = plt.subplots()[0]
```

```
clusters = kmeans.fit_predict(data.iloc[:, 1:])
```

```
data["label"] = clusters
```

```
ax = fig.add_subplot(111, projection='3d')
```

然后你可以利用 `ax.scatter()` 完成三维散点图的绘制。

实验任务 2-4 PCA 降维

实验要求

在主成分分析（PCA）实验中，我们将使用此前用过的鸢尾花数据集进行降维。通过使用 PCA 算法，我们将高维的鸢尾花数据转换为二维数据，并将其可视化展示，以便更好地理解数据的分布情况及 PCA 的原理。

希望同学们能够绘制一个散点图，展示使用 PCA 降维后的鸢尾花数据在二维平面上的分布情况。不同的花类别将使用不同颜色表示，以便观察数据的聚类情况。

实验参考结果

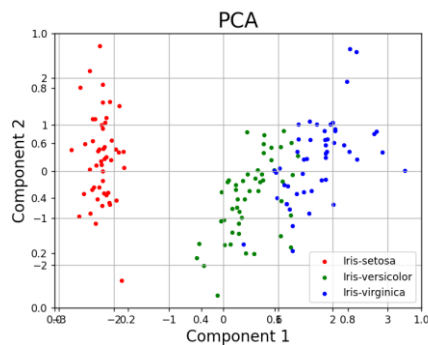


图 2-4

实验提示

- (1) 鸢尾花数据集的读取可参见本讲实验任务 2-2 的实验提示
- (2) 以下示例代码可以完成数据的标准化处理及 PCA 降维

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
data = StandardScaler().fit_transform(data) #将数据标准化处理，进行特征缩放
pca = PCA(n_components=? ) # 将特征通过 PCA 降维
components = pca.fit_transform(data)
```

- (3) 以下示例代码可实现设置目标类别和对应颜色，并根据目标类别绘制散点图
- ```
targets = [0, 1, 2]
colors = ['r', 'g', 'b']
for target, color in zip(targets, colors): # 根据目标类别用不同颜色绘制散点图
 lis = final_data['target'] == target #final_data 为降维后的二维特征（component1、
 componet2）和样本分类标签值(target)拼接而成的 DataFrame
 ax.scatter(final_data.loc[lis, 'component 1'], final_data.loc[lis, 'component 2'], c=color, s=10)
```

## 4. 参考文档

Pyplot 官方文档 [Pyplot tutorial — Matplotlib 3.7.2 documentation](https://matplotlib.org/3.7.2/tutorials/index.html)